🚀 PollingEngine.js：模块深度剖析 (v2)

🧱 第一步：业务黑盒（产品说明书视角）

1. **核心职责**：它是整个工业网关系统的**“节拍器”**。就像嵌入式里的定时器中断服务程序（TIM_IRQHandler），它负责按照预设的频率，主动敲开每一台 GD32 硬件的门（Modbus TCP 502 端口），把温湿度传感器数据拉回来，并由于 TCP 链路空闲会自动断开，它还额外负责每 5 秒发送一次心跳包来“喂狗”保活。
2. **输入**：
   - **连接池 (`DevicePool`)**：它需要知道当前的通信链路（Socket）是否建立。
   - **轮询配置 (`intervalMs`)**：设定巡检的节奏（默认 1000ms）。
3. **输出**：
   - **数据流 (`onData`)**：解析出来的 16 路温湿度、22 路继电器状态，打包成 JSON 送往前端展示。
   - **硬件指令**：构造 Modbus 0x03 (读) 和 0x10 (多写) 协议报文。
4. **使用场景举例**：
   - 在现代化养殖场，管理员在网页端看温度。`PollingEngine` 正在后台每秒扫描一次，一旦发现温度 28.5℃，立刻通过 WebSocket 广播给网页看板。
   - 管理员点击“开启风机”，`PollingEngine` 会立刻插队抢占总线（writeLock），把控制码发给 GD32 寄存器。

🦴 第二步：静态骨架（代码结构图）

核心状态与变量

- `this.isRunning`：**引擎使能位。** Boolean 值，标识轮询器是否在工作。
- `this.isPolling`：**重入保护锁。** 防止上一轮采集没结束（如网络卡顿），下一轮又发起了新的指令导致协议栈奔溃。
- `this.writeLock`：**互斥锁。** 在进行控制指令（OTA、继电器切换）时，暂停常规采集，确保指令执行的原子性。
- `this.deviceStatus`：**频率分频表 (Map)**。记录上次采集时间戳，实现“高频心跳、低频采集”的业务分流。

核心方法

- `start() / stop()`：开启/关闭定时器（类比：`Timer_Start()`）。
- `pollAllDevices()`：轮询入口，遍历连接池中的所有活动节点。
- `pollDevice(key)`：单机逻辑，处理该设备的“重连、心跳、采集”三级跳。
- `controlRelay(key, index, val)`：继电器控制逻辑（带“读-改-写”校验流程）。

⚙️ 第三步：动态运转（生命周期与数据流）

1. **启动与循环机制**：
   - 外部调用 `start()` 时，会立即执行一次全量轮询，随后挂载 `setInterval`。
   - **防死锁/防重入 (Re-entrancy Protection)**：代码中通过 `if (this.isPolling) return;` 做了硬防护。类比嵌入式，这防止了在一个低优先级的定时器中断还没退出时，

新的同级中断又抢占导致栈溢出。

2. **异常与边界处理**：
   - **网络保护**：在 `pollDevice` 中先判断状态，非 `CONNECTED` 则自动触发重连。重连后通过 `setTimeout(resolve, 300)` 预留了物理层建立连接后的平稳期。
   - **分频并发策略**：心跳 5s/次，环境 30s/次。利用 `Date.now()` 进行差值判断，这在嵌入式循环中是非常经典的多任务分期管理方式（Task Scheduling）。

🎯 第四步：执行主线

1. Happy Path（心跳与采集主线）

```
// [执行主线] 模拟一个设备的完整采集周期
async pollDevice(key) {
    const now = Date.now();
    // 步骤1: 检查心跳 (5s一次)
    if (now - lastHeartbeat >= 5000) {
        await this.pool.writeRegister(key, 0x0000, counter); // 定向喂狗
    }

    // 步骤2: 环境采集 (30s一次)
    if (now - lastSensorPoll >= 30000) {
        // 读取块并移交给 DataParser
        const raw = await this.readBlock(key, 'BLOCK_ENV');
        const parsed = this.parser.parseSensorData(raw);
        this.onData(key, parsed); // 通过回调向上层（WebSocket）投递
    }
}
```

2. Edge Cases（逆风局：重入与抢占防护）

- **重入拦截**：如果因为网络超时（Timeout）导致 `await pollAllDevices()` 阻塞时间超过 1000ms，下一轮循环会被 `if (this.isPolling)` 拦截并直接 `return`，避免总线上积压多个待处理事务。
- **互斥控制**：在 `triggerOTA` 或 `controlRelay` 期间，`writeLock` 为 true，主循环通过 `if (this.writeLock) return;` 自动空转。类比：**在临界区代码段关闭全局中断。**

3. 关键语法点拨（跨界降维打击）

> 💡 *`setInterval` 就像嵌入式里的系统节拍脉冲（Systick），驱动着整个逻辑状态机不断轮转。*
>
> 💡 *`Map` 就像是一个高效的索引结构体数组。`this.deviceStatus.set(key, ...)` 的操作，相当于用 ID 作为下标，直接访问内存里的特定状态寄存器组。*
>
> 💡 *`finally { this.isPolling = false; }` 就像 C 语言里的资源释放。无论前面的业务代码是否抛出异常（Exception/Error），这段逻辑**百分之百**会执行，保证了"保护锁"一定会被释放，不会死锁。*

📄 已归档至分析目录：

1. 可视化时序图：docs/analysis/PollingEngine/PollingEngine_diagram.html
2. 深度剖析文档：docs/analysis/PollingEngine/PollingEngine.md
3. 便携版文档：docs/analysis/PollingEngine/PollingEngine.pdf 💡 提示：VS Code 可直接预览 Markdown 和 HTML 文件，按需发送 PDF 文档。